

28 Matrix Rotate matrix by 90 degrees

// Rotate matrix by 90 degrees clockwise in-place

```
void rotate(vector<vector<int>>& mat) {  
    int n = mat.size();  
  
    // Step 1: Transpose (swap mat[i][j] with mat[j][i])  
    for (int i = 0; i < n; i++) {  
        for (int j = i + 1; j < n; j++) {  
            swap(mat[i][j], mat[j][i]);  
        }  
    }  
  
    // Step 2: Reverse each row  
    for (int i = 0; i < n; i++) {  
        reverse(mat[i].begin(), mat[i].end());  
    }  
}
```

The screenshot displays a coding platform interface. On the left, the 'Output Window' shows 'Problem Solved Successfully' with a green checkmark. Below this, statistics are listed: 'Test Cases Passed' as '1111 / 1111', 'Attempts: Correct / Total' as '1 / 1', 'Accuracy: 100%', 'Points Scored' as '4 / 4', and 'Time Taken' as '0.02'. At the bottom, there are buttons for 'Solve Next' and 'Find kth element of spiral matrix', 'Rotate by 90 degree', and 'Rotate a Matrix'. On the right, the code editor shows the C++ implementation of the rotate function, which includes comments for Step 1 (Transpose) and Step 2 (Reverse each row). The code is as follows:

```
1 // Rotate matrix by 90 degrees clockwise in-place  
2 void rotate(vector<vector<int>>& mat) {  
3     int n = mat.size();  
4  
5     // Step 1: Transpose (swap mat[i][j] with mat[j][i])  
6     for (int i = 0; i < n; i++) {  
7         for (int j = i + 1; j < n; j++) {  
8             swap(mat[i][j], mat[j][i]);  
9         }  
10    }  
11  
12    // Step 2: Reverse each row  
13    for (int i = 0; i < n; i++) {  
14        reverse(mat[i].begin(), mat[i].end());  
15    }  
16 }  
17
```